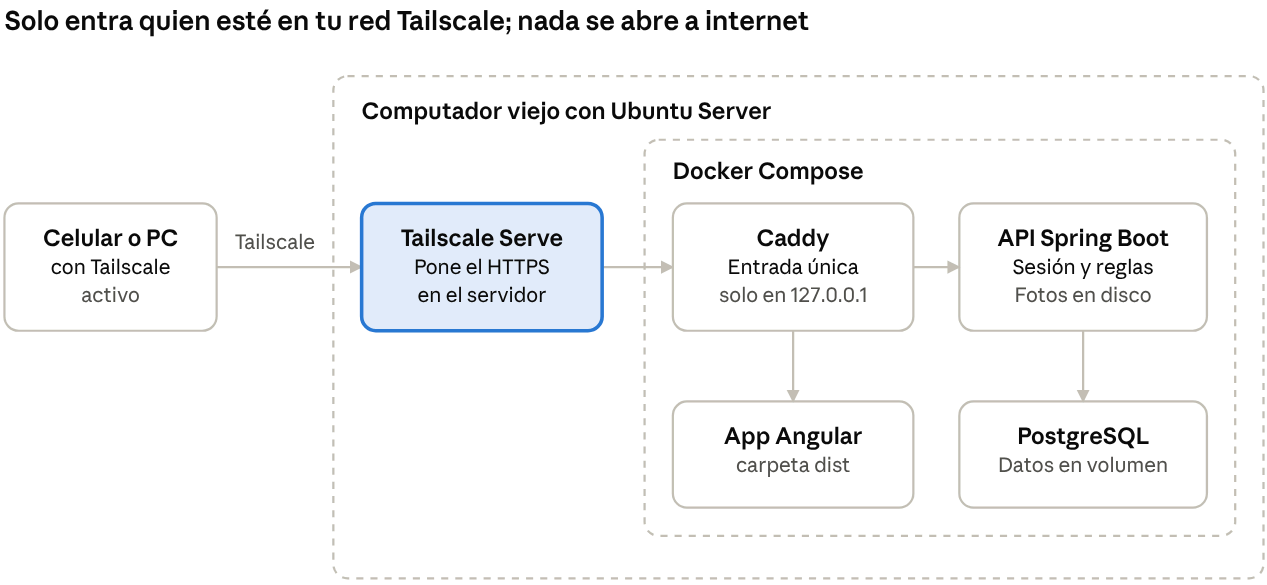
Nuestro Espacio — Despliegue en Ubuntu Server con Tailscale

Oct 27, 2026 · @Nicky Florez

1. Qué vamos a montar



cómo llega tu celular a la app · VPN, Serve, Caddy, API y base de datos

Tu celular o el de cada persona que invites se conecta por Tailscale (una VPN privada) al computador viejo. Allí, Tailscale pone el HTTPS, Caddy reparte el trabajo (entrega la app de Angular y pasa `/api` al backend) y el backend habla con la base de datos. Nada de esto se ve desde internet.

2. Reglas del despliegue

Estas son las reglas que no se negocian. Todo lo demás del documento está hecho para cumplirlas.

#	Regla	Cómo se cumple
1	La app **no es pública**	Nadie desde internet puede abrirla; solo los dispositivos de tu red Tailscale

#	Regla	Cómo se cumple
2	**No se abre ningún puerto** en el router	Tailscale conecta hacia afuera; no necesitas IP pública
3	Todo corre en el computador viejo, con Docker Compose	Base de datos, backend y Caddy son contenedores; los datos quedan en tu disco
4	El frontend es solo archivos estáticos	Angular se compila a una carpeta `dist` que Caddy sirve
5	La base de datos y el backend **no se ven desde fuera**	No publican puertos; solo se hablan entre contenedores
6	Los secretos **no van en Git**	Viven en un archivo `.env` con permisos `600`
7	La VPN **no reemplaza el login**	La app sigue pidiendo perfil y PIN (Spring Security + JWT)
8	Si el equipo se apaga y vuelve, **la app vuelve sola**	`restart: unless-stopped` y Docker habilitado al arrancar
9	Hay **respaldos cifrados**	Copia diaria de la base y las fotos a un disco externo
10	Nada de Funnel, ngrok ni túneles públicos	Solo Tailscale **Serve**, que es privado

3. Qué necesitas antes de empezar

- **El computador viejo** con al menos 4 GB de RAM y 40 GB de disco libres (la base de datos y las fotos crecen con el tiempo). Si es un portátil, debe tener el cargador conectado siempre.
- **Una memoria USB** de 8 GB o más para instalar Ubuntu Server (se borra al preparar el instalador).
- **Un disco externo** para los respaldos (puede ser pequeño al comienzo).
- **Un cable de red** del servidor al router. Es más estable que el Wi-Fi para un servidor.
- **Una cuenta de Tailscale** (gratis para uso personal). Inicia sesión con una cuenta que tenga verificación en dos pasos activada, porque esa cuenta es la llave de toda tu red privada.

- **Tu computador de trabajo** con acceso por terminal (`ssh`, `rsync`) para subir los archivos.
- **La app funcionando en tu máquina** (por lo menos hasta la fase 3 de la ruta). Este despliegue es el último paso; no tiene sentido hacerlo con una app vacía.

Nombres que usaré en los ejemplos (cámbialos por los tuyos):

Qué	Ejemplo
Usuario de Ubuntu	`nicky`
Nombre del servidor	`espacio`
Carpeta del proyecto en el servidor	`/opt/espacio`
Base de datos y usuario	`espacio`

4. Paso 1 · Preparar el computador e instalar Ubuntu Server

Qué logras: un Ubuntu Server actualizado, accesible por SSH desde tu computador de trabajo y con el firewall activo.

4.1 Instalar Ubuntu Server

1. Descarga la imagen de **Ubuntu Server LTS** desde ubuntu.com y grábala en la USB con Balena Etcher o Rufus.
2. Arranca el computador viejo desde la USB (suele ser la tecla F12, F2 o Esc al encender).
3. Durante el instalador: idioma, conexión por cable, y **marca "Install OpenSSH server"**. No instales ninguno de los paquetes opcionales que te ofrezca (Docker lo haremos a mano para tener la versión oficial).
4. Crea tu usuario (`nicky`) con una contraseña larga y pon el nombre del equipo `espacio`.

4.2 Que el equipo siempre esté disponible

- En la BIOS busca la opción **"Restore on AC Power Loss"** (o parecida) y déjala en *Power On*: así el equipo enciende solo después de un corte de luz.

- Si es un portátil, evita que se suspenda al cerrar la tapa. Edita `/etc/systemd/logind.conf`, deja `HandleLidSwitch=ignore` (sin `#` al comienzo) y reinicia el servicio:

```
sudo nano /etc/systemd/logind.conf
sudo systemctl restart systemd-logind
```

4.3 Entrar por SSH y actualizar

Busca la IP del servidor en tu router (algo como `192.168.1.50`) y desde tu computador de trabajo:

```
ssh nicky@192.168.1.50
sudo apt update && sudo apt upgrade -y
```

Copia tu llave para no usar contraseña (desde tu computador de trabajo, en otra terminal):

```
ssh-keygen -t ed25519          # solo si aún no tienes una llave
ssh-copy-id nicky@192.168.1.50
```

Cuando entres sin que te pida contraseña, desactiva el ingreso con contraseña en el servidor:

```
sudo nano /etc/ssh/sshd_config.d/99-seguro.conf
```

con este contenido:

```
PasswordAuthentication no
PermitRootLogin no
```

```
sudo systemctl restart ssh
```

4.4 Firewall y actualizaciones automáticas

Permite SSH primero y recién después activa el firewall; al revés te dejas fuera del servidor.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

Tu app no necesita ningún puerto abierto aquí: Tailscale la alcanza por su propio camino. Las actualizaciones de seguridad se instalan solas con:

```
sudo dpkg-reconfigure -plow unattended-upgrades
```

Prueba de salida: `ssh` entra sin contraseña, `sudo ufw status` dice *active*, y `sudo apt update` no da errores.

5. Paso 2 · Instalar Docker y Docker Compose

Qué logras: poder levantar contenedores en el servidor con un solo comando.

Instala Docker desde su repositorio oficial (las líneas son largas; cópialas completas). Copia y pega bloque por bloque:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

Permite usar Docker sin escribir `sudo` cada vez, y **cierra y vuelve a abrir la sesión SSH** para que el cambio aplique:

```
sudo usermod -aG docker $USER
exit
```

Vuelve a entrar por SSH y prueba:

```
docker --version
docker compose version
docker run --rm hello-world
```

Docker ya queda activado para arrancar con el equipo. Comprueba con `systemctl is-enabled docker` (debe decir `enabled`).

Prueba de salida: `hello-world` imprime el mensaje de bienvenida sin `sudo`.

Ojo: quien está en el grupo `docker` puede hacer cualquier cosa en el servidor. En un equipo tuyo, solo contigo, es aceptable; no agregues a nadie más.

6. Paso 3 · Instalar Tailscale en el servidor

Qué logras: que el servidor entre a tu red privada de Tailscale y sea alcanzable desde tus dispositivos, sin abrir nada en el router. Tailscale se instala directo en Ubuntu, **no** en Docker.

1. Instala y conecta el servidor a tu cuenta:

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --hostname=espacio
```

2. El comando muestra un enlace. Ábrelo en el navegador de tu computador de trabajo e inicia sesión para aprobar el servidor.
3. Instala la app de **Tailscale en tu celular** e inicia sesión con la misma cuenta.
4. Verifica desde el servidor:

```
tailscale status
tailscale ip -4
```

Debes ver `espacio` y tu celular en la lista. Desde el celular, con Tailscale activo, ya puedes abrir un `ping` o una conexión al servidor.

5. **Muy importante:** entra a la consola de administración de Tailscale (login.tailscale.com), pestaña *Machines*, menú `···` del servidor, y elige **"Disable key expiry"**. Si no lo haces, el servidor se desconecta de tu red cada ciertos meses y la app queda inaccesible sin aviso.

6. En la pestaña *DNS* de la consola, activa **MagicDNS** y **HTTPS Certificates**. Los necesitas para el paso 10.

Prueba de salida: `tailscale status` muestra el servidor y tu celular conectados; la clave del servidor ya no tiene fecha de expiración.

Privacidad: el nombre del equipo (`espacio`) quedará visible en los registros públicos de certificados cuando actives HTTPS. Por eso no uses tu nombre ni el de otras personas como nombre del servidor.

7. Paso 4 · Carpetas del proyecto y archivo de secretos

Qué logras: dejar lista la carpeta donde vivirá todo y guardar las contraseñas fuera del código.

```
sudo mkdir -p /opt/espacio
sudo chown $USER:$USER /opt/espacio
cd /opt/espacio
mkdir -p backend frontend data/uploads
```

Así quedará la carpeta cuando termines:

```
/opt/espacio
├── docker-compose.yml     # paso 6
├── Caddyfile              # paso 7
├── .env                   # secretos, nunca en Git
├── backup.sh              # paso 11
├── backend/               # código de Spring Boot + Dockerfile
├── frontend/              # archivos compilados de Angular (dist)
└── data/
    └── uploads/           # fotos y portadas, una subcarpeta por sala
```

La carpeta de fotos la usará el backend con un usuario interno (número 1001, que se crea en el Dockerfile del paso 5). Dale permiso de escritura a ese número:

```
sudo chown -R 1001:1001 /opt/espacio/data/uploads
```

Crea el archivo `.env`. Genera los secretos con el propio servidor, no los inventes a mano:

```
cd /opt/espacio
echo "DB_PASSWORD=$(openssl rand -base64 24 | tr -d '/+=')" > .env
echo "JWT_SECRET=$(openssl rand -base64 48 | tr -d '\n')" >> .env
chmod 600 .env
nano .env
```

Complétalo para que quede así (los valores de ejemplo son inventados):

```
DB_NAME=espacio
DB_USER=espacio
DB_PASSWORD=<ya generado>
JWT_SECRET=<ya generado>
APP_ORIGIN=https://espacio.TU-RED.ts.net
SEED_ADMIN_USER=<tu usuario>
SEED_ADMIN_PASSWORD=<contraseña larga>
SEED_SECOND_USER=<usuario de la segunda persona>
SEED_SECOND_PASSWORD=<contraseña temporal>
SEED_ROOM_NAME=<nombre de la sala sembrada>
VAPID_PUBLIC=<paso de notificaciones>
VAPID_PRIVATE=<paso de notificaciones>
```

`APP_ORIGIN` lo cambias en el paso 10, cuando veas tu dirección real de Tailscale. Los nombres `SEED_*` y `VAPID_*` deben coincidir con los que tu código lea en `application.yml`; si usaste otros nombres, usa los tuyos.

Prueba de salida: `ls -l .env` muestra `-rw-------` (solo tú puedes leerlo) y `data/uploads` pertenece al usuario 1001.

8. Paso 5 · Dockerfile del backend

Qué logras: convertir tu proyecto Spring Boot en una imagen que Docker sabe construir y ejecutar. Guarda este archivo como `backend/Dockerfile` en tu proyecto.

```
# --- Etapa 1: compilar el .jar ---
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests
```

```
# --- Etapa 2: imagen final, liviana ---
FROM eclipse-temurin:21-jre
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/* \
 && useradd -r -u 1001 app \
 && mkdir -p /data/uploads \
 && chown app /data/uploads
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Qué hace cada parte:

Línea	Para qué sirve
Dos etapas (`AS build` y la final)	La imagen final solo lleva Java y tu `.jar`, sin Maven ni código fuente: es más pequeña y más segura
`COPY pom.xml` y luego `dependency:go-offline`	Docker guarda en caché las dependencias; las siguientes compilaciones son mucho más rápidas si no cambias el `pom.xml`
`useradd -r -u 1001 app` y `USER app`	El backend **no corre como administrador**; si alguien lo vulnerara, tendría muy pocos permisos
`curl`	Lo usa la verificación de salud del paso 6
`/data/uploads` con dueño `app`	Carpeta donde se guardan las fotos

Añade también un archivo `backend/.dockerignore` para no copiar basura:

```
target
.git
*.md
```

Prueba de salida (en tu computador de trabajo): `docker build -t espacio-api ./backend` termina sin errores. Si falla aquí, fallará igual en el servidor, así que es mejor descubrirlo en tu máquina.

9. Paso 6 · El `docker-compose.yml`

Qué logras: un solo archivo que describe los tres contenedores (base de datos, backend y Caddy), cómo se hablan y qué datos conservan. Guárdalo en `/opt/espacio/docker-compose.yml`.

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks: [db_net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d
$${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  api:
    build: ./backend
    restart: unless-stopped
    env_file: .env
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/${DB_NAME}
      JAVA_TOOL_OPTIONS: "-Xmx512m"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./data/uploads:/data/uploads
    networks: [db_net, web_net]
    read_only: true
    tmpfs:
      - /tmp
    cap_drop: [ALL]
    security_opt:
      - no-new-privileges:true
```

```
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8080/actuator/health"]
      interval: 15s
      timeout: 5s
      retries: 5
      start_period: 60s

  caddy:
    image: caddy:2
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:80"
    depends_on:
      api:
        condition: service_healthy
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - ./frontend:/srv:ro
      - caddy_data:/data
      - caddy_config:/config
    networks: [web_net]
    cap_drop: [ALL]
    cap_add: [NET_BIND_SERVICE]
    security_opt:
      - no-new-privileges:true

networks:
  web_net: {}
  db_net:
    internal: true

volumes:
  pgdata:
  caddy_data:
  caddy_config:
```

Las líneas que importan, explicadas:

Línea	Qué significa
`ports: "127.0.0.1:8080:80"`	**La línea más importante.** Caddy solo escucha dentro del propio servidor. Sin el `127.0.0.1`, Docker abriría el puerto a toda tu red de casa y ni siquiera el firewall lo frenaría
`db` y `api` sin `ports`	No publican nada: solo otros contenedores pueden hablarles
`db_net` con `internal: true`	La base de datos vive en una red sin salida a internet. Solo la API puede alcanzarla
`api` en `db_net` y `web_net`	La API habla con la base de datos por una red y con Caddy por la otra. También necesita salida a internet para consultar Spotify o YouTube
`volumes: pgdata`	Los datos de PostgreSQL sobreviven aunque borres o recrees el contenedor
`./data/uploads:/data/uploads`	Las fotos quedan en una carpeta normal de tu disco, fácil de respaldar
`restart: unless-stopped`	Si el contenedor se cae o el equipo se reinicia, Docker lo vuelve a levantar
`healthcheck` + `depends_on: service_healthy`	La API espera a que la base de datos esté lista, y Caddy espera a la API
`read_only`, `cap_drop`, `no-new-privileges`	Cierran permisos que la API no necesita. Si algún día la API falla por no poder escribir en un archivo, revisa los registros antes de quitar estas líneas
`JAVA_TOOL_OPTIONS -Xmx512m`	Limita la memoria de Java para que un computador viejo no se ahogue
`$${POSTGRES_USER}` (dos `$`)	Obliga a que la variable la lea el contenedor y no Compose

Prueba de salida: más adelante, `docker compose config -q` no imprime nada (sin errores) cuando el archivo está bien escrito.

10. Paso 7 · El Caddyfile

Qué logras: que Caddy entregue los archivos de Angular y reenvíe todo lo que empiece con `/api` al backend. Guárdalo en `/opt/espacio/Caddyfile`.

Caddy normalmente pide certificados HTTPS por su cuenta, pero aquí **no** lo hace: el HTTPS lo pone Tailscale en el paso 10, así que Caddy trabaja en HTTP sencillo, solo dentro del servidor.

```
{
        auto_https off
        admin off
}

:80 {
        encode gzip

        request_body {
                max_size 45MB
        }

        header {
                X-Content-Type-Options nosniff
                Referrer-Policy strict-origin-when-cross-origin
                -Server
        }

        @nocache path /ngsw-worker.js /ngsw.json /index.html
/manifest.webmanifest
        header @nocache Cache-Control "no-cache"

        handle /actuator/* {
                respond "No encontrado" 404
        }

        handle /api/* {
                reverse_proxy api:8080
        }

        handle {
                root * /srv
```

```
                try_files {path} /index.html
                file_server
        }
}
```

Bloque	Para qué sirve
`auto_https off` y `admin off`	Caddy no intenta sacar certificados ni abre su panel interno
`request_body max_size 45MB`	Mismo tope que tu `max-request-size` de Spring (paso de fotos); si subes más fotos juntas, súbelo en los dos sitios
`@nocache`	Esos archivos controlan las actualizaciones de la PWA. Si el navegador los guarda en caché, tus celulares tardan días en ver una versión nueva
`handle /actuator/*`	La verificación de salud solo la usa Docker por dentro; nadie más necesita verla
`handle /api/*`	Todo lo que empiece con `/api` va a Spring Boot (`api` es el nombre del servicio en el Compose)
`try_files {path} /index.html`	Si abres `/photos/3` y recargas, Caddy devuelve la app de Angular en vez de un error 404

En Spring Boot añade esta línea a `application.yml` para que sepa que la petición original llegó por HTTPS aunque Caddy le hable en HTTP:

```
server:
  forward-headers-strategy: framework
```

Si cambias el Caddyfile después, como el panel interno está apagado, recarga con `docker compose restart caddy`.

Prueba de salida: se hace en el paso 9, cuando todo esté arriba.

11. Paso 8 · Subir el backend y el frontend al servidor

Qué logras: copiar al servidor el código del backend, los archivos compilados de Angular y los dos archivos de configuración. Todo se hace **desde tu computador de trabajo**, en la carpeta raíz de tu proyecto (la que tiene `backend/` y `frontend/`).

Usa `nicky@espacio` si tu computador ya tiene Tailscale y MagicDNS; si no, usa la IP de tu red de casa (`nicky@192.168.1.50`).

1. Copiar la configuración:

```
scp docker-compose.yml Caddyfile nicky@espacio:/opt/espacio/
```

2. Copiar el backend (sin la carpeta `target`, que pesa mucho y no hace falta):

```
rsync -av --delete --exclude target --exclude .git backend/
nicky@espacio:/opt/espacio/backend/
```

3. Compilar el frontend y copiarlo. Angular genera la carpeta de salida dentro de `dist/<nombre-del-proyecto>/browser` (revisa el nombre exacto después de compilar):

```
cd frontend
npm ci
npx ng build
cd ..
rsync -av --delete frontend/dist/nuestro-espacio/browser/
nicky@espacio:/opt/espacio/frontend/
```

La app llama a la API con rutas relativas (`/api/v1/...`), así que **no necesitas cambiar nada** entre tu máquina y el servidor. Si en algún sitio escribiste `http://localhost:8080`, cámbialo por una ruta relativa antes de compilar.

Prueba de salida: en el servidor, `ls /opt/espacio/frontend` muestra `index.html`, `ngsw.json` y los `.js`; `ls /opt/espacio/backend` muestra `Dockerfile` y `pom.xml`.

Atención con `--delete`: borra en el destino lo que no exista en el origen. Revisa dos veces que la carpeta de destino termine en `/backend/` o `/frontend/`; nunca la raíz de `/opt/espacio`, porque borraría tus fotos y tu `.env`.

12. Paso 9 · Levantar todo y probarlo

Qué logras: ver la app corriendo en el servidor y confirmar que solo se puede alcanzar desde donde debe.

En el servidor (por SSH):

```
cd /opt/espacio
docker compose config -q          # valida el archivo; sin salida = bien
docker compose up -d --build      # construye y levanta (la primera vez tarda
varios minutos)
docker compose ps                 # los tres deben decir "healthy" o
"running"
docker compose logs -f api        # Ctrl+C para salir
```

En los registros de la API debes ver que Flyway aplicó las migraciones y que Spring Boot arrancó ("Started … in … seconds"). Si algo falla, el error aparece ahí.

Pruebas, una por una:

Prueba	Comando	Resultado esperado
La web responde (desde el servidor)	`curl -I http://127.0.0.1:8080/`	`HTTP/1.1 200 OK`
La API responde	`curl -s http://127.0.0.1:8080/api/v1/profiles`	Un JSON con los dos perfiles
Se ven solo los puertos correctos	`ss -tlnp | grep -E '8080|5432'`	`127.0.0.1:8080`, y nada en `5432`
Desde otro dispositivo de tu casa, la web **no** abre	En el navegador de otro equipo: `http://192.168.1.50:8080`	No conecta
La base de datos no es alcanzable desde afuera	`nc -zv 192.168.1.50 5432` desde otro equipo	Conexión rechazada
Los datos sobreviven un reinicio	`sudo reboot`, esperar 2 minutos y repetir la segunda prueba	Responde igual, sin que hagas nada

La última prueba es la más importante: demuestra que, si se va la luz, el servidor vuelve solo.

Prueba de salida: las seis pruebas dan el resultado esperado.

13. Paso 10 · Tailscale Serve (HTTPS) y compartirlo con más personas

Qué logras: una dirección `https://...` que solo abre desde dispositivos autorizados de tu red Tailscale. Las PWA necesitan HTTPS para instalarse y recibir notificaciones, y Tailscale Serve lo pone sin que compres un dominio.

Serve es privado, Funnel es público. Aquí solo usamos `serve`. No uses `funnel` nunca.

13.1 Activarlo

Asegúrate de haber activado MagicDNS y HTTPS Certificates en el paso 3. Luego, en el servidor:

```
sudo tailscale serve --bg 8080
tailscale serve status
```

El comando reenvía el HTTPS de tu dirección de Tailscale hacia `127.0.0.1:8080`, donde escucha Caddy. El segundo comando muestra tu dirección, algo como `https://espacio.tu-red.ts.net`. La sintaxis de `serve` ha cambiado entre versiones; si tu versión no acepta el comando tal cual, mira `tailscale serve --help`. La configuración con `--bg` se conserva después de reiniciar.

13.2 Decirle a la app cuál es su dirección

Edita `/opt/espacio/.env` y pon tu dirección real (sin barra final):

```
APP_ORIGIN=https://espacio.tu-red.ts.net
```

```
cd /opt/espacio
docker compose up -d
```

13.3 Probar desde tu celular

1. En el celular, **apaga el Wi-Fi** (usa datos móviles) y confirma que Tailscale está activo.

2. Abre la dirección en el navegador: debe aparecer la pantalla de perfiles.
3. Instala la app: en Android, menú del navegador → *Instalar aplicación*; en iPhone, Safari → *Compartir* → *Agregar a pantalla de inicio*.
4. Apaga Tailscale en el celular y recarga: **no debe abrir**. Eso confirma que la app es privada.

13.4 Darle acceso a otras personas

La forma más limpia es **compartir solo el servidor** con su cuenta, para que esa persona no vea tus otros dispositivos:

1. En la consola de Tailscale, *Machines*, menú `⋯` del servidor → *Share* → genera el enlace de invitación.
2. Cada persona crea su propia cuenta de Tailscale (gratis), abre el enlace y acepta.
3. Instala Tailscale en su celular, lo activa y abre la dirección `https://...` que le envíes.
4. Para quitarle el acceso en cualquier momento, deja de compartir el servidor desde la misma pantalla.

No lo probé en tu red, así que valida en este orden: que la persona vea el servidor, que abra la dirección `https` y que pueda entrar con su usuario. Si la dirección no abre desde su cuenta, la alternativa es invitarla como usuaria de tu red (*Settings* → *Users*) y limitar lo que ve con las reglas de acceso (*Access controls*) de la consola.

Lo que cada persona debe saber: para abrir la app, Tailscale tiene que estar activo en su celular. Si el icono de la VPN no aparece, la app no cargará. Es el precio de que nadie más pueda entrar.

Prueba de salida: otra persona inicia sesión con su usuario desde su celular, fuera de tu casa.

Límite a vigilar: cada persona nueva necesita acceso a tu tailnet. Antes de invitar a un grupo grande revisa los límites de usuarios y dispositivos de tu plan de Tailscale; si la app crece, es el primer cuello de botella. Las salas (y quién entra a cada una) se manejan dentro de la app; Tailscale solo decide quién llega al servidor.

14. Paso 11 · Respaldos automáticos cifrados

Qué logras: que cada noche se guarde una copia cifrada de la base de datos, las fotos y tu `.env` en un disco externo. Usamos `restic`: cifra, no repite lo que ya copió y borra las copias viejas solo.

Importante: un servidor con un solo disco no tiene respaldo. Si ese disco falla, pierdes todo. El disco externo es obligatorio, no opcional.

14.1 Preparar el disco externo

Conecta el disco y busca su nombre:

```
lsblk -f
```

Si el disco ya tiene datos que quieres conservar, **no sigas con el formateo**. Si está vacío o no te importa borrarlo, fórmatealo (cambia `sdb1` por el nombre real; si te equivocas, borras el disco equivocado):

```
sudo mkfs.ext4 -L respaldo /dev/sdb1
```

Haz que se monte solo en cada arranque. Mira el `UUID` con `lsblk -f` y añade esta línea al final de `/etc/fstab` (`sudo nano /etc/fstab`):

```
UUID=PON-AQUI-EL-UUID  /mnt/respaldo  ext4  defaults,nofail  0  2
```

```
sudo mkdir -p /mnt/respaldo
sudo mount -a
df -h /mnt/respaldo
```

14.2 Crear el repositorio cifrado

```
sudo apt install -y restic
sudo sh -c 'openssl rand -base64 32 > /root/.restic-pass && chmod 600
/root/.restic-pass'
sudo cat /root/.restic-pass
sudo restic init --repo /mnt/respaldo/espacio --password-file /root/.restic-
pass
```

Copia la contraseña que imprimió `cat` en tu gestor de contraseñas, ahora. Sin ella, los respaldos no se pueden abrir, y si el servidor se daña tampoco tendrás la copia de la contraseña.

14.3 El script de respaldo

Créalo en `/opt/espacio/backup.sh` :

```
#!/bin/bash
set -euo pipefail
umask 077
cd /opt/espacio
export RESTIC_REPOSITORY=/mnt/respaldo/espacio
export RESTIC_PASSWORD_FILE=/root/.restic-pass

mountpoint -q /mnt/respaldo || { echo "Disco de respaldo no montado"; exit 1;
}

docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"'
> /root/espacio.sql
restic backup /root/espacio.sql /opt/espacio/data/uploads /opt/espacio/.env
rm -f /root/espacio.sql
restic forget --keep-daily 14 --keep-weekly 8 --prune
```

```
sudo chmod 700 /opt/espacio/backup.sh
sudo /opt/espacio/backup.sh
sudo restic -r /mnt/respaldo/espacio --password-file /root/.restic-pass
snapshots
```

Si ves una línea de copia en `snapshots`, funciona. Para que corra sola cada noche a las 3:30, abre el cron de root con `sudo crontab -e` y añade:

```
30 3 * * * /opt/espacio/backup.sh >> /var/log/espacio-backup.log 2>&1
```

14.4 La prueba que no puedes saltarte: restaurar

Una copia que nunca restauraste no es una copia. Hazlo hoy, y de nuevo cada pocos meses:

```
sudo restic -r /mnt/respaldo/espacio --password-file /root/.restic-pass
restore latest --target /tmp/restauracion
cd /opt/espacio
docker compose exec -T db sh -c 'createdb -U "$POSTGRES_USER"
prueba_restauracion'
cat /tmp/restauracion/root/espacio.sql | docker compose exec -T db sh -c
'psql -q -U "$POSTGRES_USER" prueba_restauracion'
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" prueba_restauracion
-c "\dt"'
```

Si `\dt` lista tus tablas, la copia es buena. Limpia lo de la prueba:

```
docker compose exec -T db sh -c 'dropdb -U "$POSTGRES_USER"
prueba_restauracion'
sudo rm -rf /tmp/restauracion
```

Prueba de salida: `snapshots` muestra copias, y la restauración de prueba muestra tus tablas.

Límite: el disco externo vive en la misma casa. Un incendio o un robo se llevan servidor y copia. Si las fotos importan mucho, una vez al mes lleva una segunda copia del repositorio (`/mnt/respaldo/espacio`) a otro lugar; como está cifrada, es seguro.

15. Mantenimiento, problemas comunes y lista final

15.1 Cómo actualizar la app cuando cambies algo

Desde tu computador de trabajo, según lo que cambiaste:

Qué cambiaste	Qué haces
Solo el frontend	Compilas, haces el `rsync` del frontend (paso 8). No hay que reiniciar nada. Los celulares toman la versión nueva al abrir la app una o dos veces
El backend	`rsync` del backend y, en el servidor: `cd /opt/espacio && docker compose up -d --build api`
El Compose o el Caddyfile	`scp` de los archivos y `docker compose up -d` ; si solo fue el Caddyfile, `docker compose restart caddy`
Una variable del `.env`	Edita el archivo y `docker compose up -d` (recrea lo que cambió)

Antes de cada actualización del backend que toque la base de datos (una migración nueva), corre `sudo /opt/espacio/backup.sh` por si algo sale mal.

15.2 Comandos que vas a usar siempre

```
cd /opt/espacio
docker compose ps                    # estado de los tres contenedores
docker compose logs --tail 100 api # últimas líneas del backend
docker compose restart api          # reiniciar solo el backend
docker compose down                 # apagar todo (los datos se conservan)
docker compose up -d                # encender todo
df -h                               # espacio en disco
docker system prune                 # limpia imágenes viejas (te pide
confirmar)
```

`docker compose down -v` **borra los volúmenes, incluida la base de datos.** No lo uses nunca salvo que quieras empezar de cero.

15.3 Problemas comunes

Síntoma	Causa probable	Qué hacer
El navegador muestra 502	La API aún arranca o se cayó	`docker compose logs api` y revisa el error
El celular no abre la dirección	Tailscale está apagado en el celular	Activar Tailscale; revisar que el servidor aparezca conectado
Falla al subir una foto	La carpeta `data/uploads` no es del usuario 1001	`sudo chown -R 1001:1001 /opt/espacio/data/uploads`
Falla al subir varias fotos a la vez	Pasaste el tope de tamaño	Sube `max_size` en Caddy y `max-request-size` en Spring
Entra pero se cierra la sesión sola	`APP_ORIGIN` no coincide con la dirección real	Corrige el `.env` y `docker compose up -d`
La API no arranca: "password authentication failed"	Cambiaste `DB_PASSWORD` después de crear la base	PostgreSQL guarda la contraseña inicial en el volumen; cámbiala dentro de la base o restaura el valor del `.env`
La API reinicia y dice que no puede escribir un archivo	Choca con `read_only`	Revisa el registro; si el archivo es legítimo, agrégale un `tmpfs` o un volumen

Síntoma	Causa probable	Qué hacer
Los celulares no ven la versión nueva	Caché de la PWA	Cierra y abre la app dos veces; verifica que el `@nocache` del Caddyfile esté puesto
El servidor deja de aparecer en Tailscale	La clave del servidor expiró	Consola de Tailscale → *Disable key expiry*, y `sudo tailscale up` en el servidor
Disco lleno	Imágenes viejas de Docker o muchas fotos	`docker system prune` y revisa `df -h`

15.4 Lista de comprobación final

- [ ] No hay puertos redirigidos en el router
- [ ] `ufw` está activo y solo permite SSH
- [ ] El ingreso por SSH es solo con llave
- [ ] Caddy escucha en `127.0.0.1:8080`, no en `0.0.0.0`
- [ ] PostgreSQL y la API no publican ningún puerto
- [ ] `.env` tiene permisos `600` y no está en Git
- [ ] La clave del servidor en Tailscale no expira
- [ ] Usaste `tailscale serve`, nunca `funnel`
- [ ] Con Tailscale apagado en el celular, la app no abre
- [ ] Reiniciaste el servidor y la app volvió sola
- [ ] Hay una copia nocturna y probaste restaurarla
- [ ] La contraseña de `restic` está guardada fuera del servidor
- [ ] Otra persona puede entrar desde su celular, fuera de tu casa
- [ ] La cuenta de Tailscale tiene verificación en dos pasos

Opcionales, para cuando todo lo anterior esté estable: cifrar el disco del servidor (protege las fotos si lo roban, pero te obliga a escribir una clave en cada reinicio) y activar *Tailnet Lock* en Tailscale (impide que alguien agregue dispositivos aunque robe tu cuenta, pero cada dispositivo nuevo requiere una firma tuya).